GeoPKI

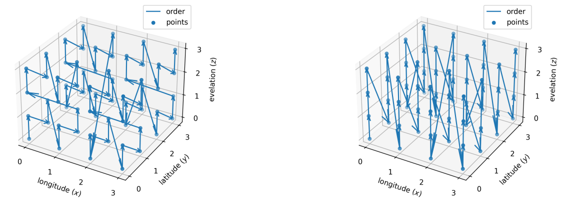
I. Model

The earth ellipsoid model of the existing world geodetic system (WGS) 84 [1] is used. WGS 84 defines the *semi-major axis* (half of the largest diameter of an ellipsoid, at the equator) a to be $6'378'137.0$ meters [1]. The *semi-minor axis* (the polar radius) b is defined as $6'356'752.3142$ meters [1]. Standard polar coordinates (longitude, latitude) are used for referencing points on the planet's two-dimensional surface and a third coordinate, elevation, extends this to three dimensions. Longitude, latitude and elevation together refer to a unique point in three-dimensions.

Each of the three coordinates is discretized in a way that preserves meter-precision in the worst case. With the semi-major axis a, the maximum circumference of the ellipsoid equals $C_x := 2a\pi$ meters. Using base two, $B_x := \lceil \log_2(C_x) \rceil = 26$ bits for the longitude thus suffice to represent each possible meter value in the worst case. For the latitude we have $C_y := 2b\pi$ meters and thus $B_y := \lceil \log_2(C_y) \rceil = B_x =: B_{x,y}$, the same minimum number of bits are required. For elevation there is no natural bound so one has to be fixed. At the time of writing, the highest point on earth is below $10'000$ meters [] and the lowest layer of earth's atmosphere, the troposphere, extends to about 18 km [2]. The Mariana Trench at about $10'000$ m below sea level is likely a bound not exceeded in depth in the near future. The necessity of allowing volumes below sea level arises from countries such as the Netherlands where parts of the landmass is below sea level. Moreover in the future, below sea level research stations might be built. Thus the range starting at a depth of $D := -10'000$ m up to an elevation of $H := 20'000$ m above sea level seams reasonable and requires $B_z := \lceil \log_2(H - D) \rceil = 15$ bits.

II. Data Structure

Inspired by F-PKI [3], sparse merkle hash tree (MHT) is used. Each leaf represents a full discretized coordinate using $B := 2 \cdot B_{x,y} + B_z = 68$ bits resulting in a total of 2^B leaves. To have spatially close points somewhat close together in the MHT, the space-filling Z-order curve is used to sort the three dimensional locations in one dimension. An example with $B_{x,y} = B_z = 2$ is shown in Figure 1a. GeoPKI uses a slightly modified 3D Z-order curve: Rather than using the full 3D Z-order curve, a 2D Z-order curve over the longitude and latitude is used and points with the same longitude and latitude but different elevation are put in sequence. While this leads to spatially close points being further apart in the 1-dimensional sorting, this modification allows for shorter proofs in a majority of envisioned use-cases (will be discussed in Section II-E) and even results in a nicer binary representation. Figure 1b contrasts this modification with the standard 3D Z-order curve shown in Figure 1a.



(a) 3D Z-Order Curve

(b) 2D Z-Order Curve with elevation in sequence

Fig. 1: Difference between a standard 3D Z-order curve and the one used by GeoPKI when using two bits for all coordinates.

A. Leaf Binary Format

Each leaf of the MHT corresponds to a point and the leaves are sorted lexicographically by their binary representation in ascending order. A point tuple $p := (x, y, z)$ at longitude x, latitude y and elevation z with

$$(x, y, z) \in [-180, 180] \times [-90, 90] \times [D, H]$$

is first discretized to integer coordinates as follows:

$$\begin{aligned} p' &:= (x_d, y_d, z_d) \\ &= \left(\left\lfloor \frac{x + 180}{360} \cdot C_x \right\rfloor, \left\lfloor \frac{y + 90}{180} \cdot C_y \right\rfloor, \lfloor z - D \rfloor \right) \end{aligned} \tag{1}$$

After this transformation the following folds:

$$p' \in [0, 2^{B_x}) \times [0, 2^{B_y}) \times [0, 2^{B_z})$$

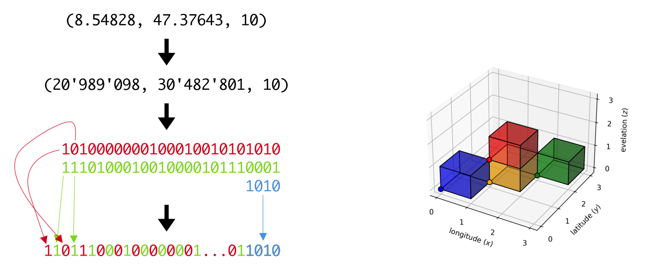
This formula holds since the first term in the longitude and latitude tuples can be at most 1 and since $C_x < 2^{B_{x,y}}$, $C_y < 2^{B_{x,y}}$ and $H < 2^{B_z}$ hold. The three resulting integer numbers can now be represented using $B_{x,y}$, $B_{x,y}$ and B_z bits, respectively. Hence let $p'' := (x_b, y_b, z_b)$ with

$$p'' \in \{0,1\}^{B_{x,y}} \times \{0,1\}^{B_{x,y}} \times \{0,1\}^{B_z}$$

For $c \in \{x, y, z\}$ and for any integer $i \in [0, 2^{B_c})$, let $c_{b,i}$ denote the i-th bit of c_b. Moreover let $||$ denote the concatenation operator. The binary representation p_b of the point p is then defined as

$$p_b = \left(\mathop{\Big|\Big|}_{i=0}^{B_{xy}-1} x_{b,i} || y_{b,i} \right) || \left(\mathop{\Big|\Big|}_{i=0}^{B_z-1} z_{b,i} \right)$$

In words, the binary representation of p is the interleaving of the binary representation for the longitude and latitude concatenated to the binary representation of the elevation. Figure 2a shows an example of this binary encoding.



(a) Binary Representation of a (longitude, latitude, elevation) tuple.

(b) Geometric Representation

Fig. 2: The binary and geometric representation of a leaf

B. Leaf Geometric Representation

Due to the floor functions in Equation (1), the point p'' represents the volume where p'' is in the corner with the smallest coordinates in all three dimensions. By replacing the floor functions by mathematically rounding or the ceil function the volumetric representation changes. In case of the floor functions shown in Equation (1), the point corresponding to p'' is

$$\left(x_d \cdot \frac{360}{C_x} - 180, x_d \cdot \frac{180}{C_y} - 90, z_d + D\right) \tag{2}$$

and the volume represented by point p'' is given by

$$\begin{aligned}&\left[x_d \cdot \frac{360}{C_x} - 180, (x_d + 1) \cdot \frac{360}{C_x} - 180\right) \times \\ &\left[y_d \cdot \frac{180}{C_y} - 90, (y_d + 1) \cdot \frac{180}{C_y} - 90\right) \times \\ &[z_d + D, z_d + D + 1)\end{aligned} \tag{3}$$

Figure 2b shows the points and the corresponding volumes in the same color.

C. Intermediate Nodes

Intermediate nodes in the MHT also correspond to a point and a volume following the same pattern as for the leaf nodes. The binary string corresponding to an intermediate node is simply the longest common prefix of it's two children and covers the whole volume of its two children. The left child is the node with the binary representation equal to the intermediate node one's concatenated with a zero and the right child concatenated with a one. The formula for computing the volume becomes more complex for intermediate nodes as it requires a case distinction: Let s be the binary string of an intermediate node.

Case 1: Assume length of s is exactly $2 \cdot B_{x,y}$. Then the corresponding x_d and y_d are well defined as for the leaves. The corresponding point is

$$\left(x_d \cdot \frac{360}{C_x} - 180, y_d \cdot \frac{180}{C_y} - 90, 0\right) \tag{4}$$

and the covered volume is given by

$$\begin{aligned}&\left[x_d \cdot \frac{360}{C_x} - 180, (x_d + 1) \cdot \frac{360}{C_x} - 180\right) \times \\ &\left[y_d \cdot \frac{180}{C_y} - 90, (y_d + 1) \cdot \frac{180}{C_y} - 90\right) \times \\ &[D, H)\end{aligned} \tag{5}$$

Case 2: Assume length of s is strictly greater than $2 \cdot B_{x,y}$. Then the corresponding x_d and y_d are also well defined as for the leaves. Let s_z be the remaining bits after the first $2 \cdot B_{x,y}$, let l be it's length and let $s_{z,i}$ for $i \in \{0, ..., l-1\}$ be the i-th bit of s_z. The point corresponding to this bit string is

$$\left(x_d \cdot \frac{360}{C_x} - 180, y_d \cdot \frac{180}{C_y} - 90, D + \sum_{i=0}^{l-1} 2^{B_z - 1 - i} \cdot s_{z,i}\right) \tag{6}$$

and the covered volume is given by

$$\begin{aligned}&\left[x_d \cdot \frac{360}{C_x} - 180, (x_d + 1) \cdot \frac{360}{C_x} - 180\right) \times \\ &\left[y_d \cdot \frac{180}{C_y} - 90, (y_d + 1) \cdot \frac{180}{C_y} - 90\right) \times \\ &\left[D + \sum_{i=0}^{l-1} 2^{B_z - 1 - i} \cdot s_{z,i}, D + \left(\sum_{i=0}^{l-1} 2^{B_z - 1 - i} \cdot s_{z,i}\right) + 2^{B_z - l}\right)\end{aligned} \tag{7}$$

Case 3: Assume length of s is strictly smaller than $2 \cdot B_{x,y}$. In this case we only have partial bits for the longitude and latitude but certainly know that the full z dimension is covered. Let l be the length of s. Let s_x and s_y be the bit prefixes encoded in s corresponding to the longitude and latitude, respectively, let l_x and l_y be the lengths of s_x and s_y, respectively, and let $s_{x,i}$ and $s_{y,j}$ for $i \in \{0, ..., l_x - 1\}, j \in \{0, ..., l_y - 1\}$ correspond to the i-th and j-th bit of s_x and s_y, respectively. It holds that $l = l_x + l_y$ and it is possible that l_x is by exactly one larger than l_y, otherwise $l_x = l_y$. This directly follows from the interleaved binary encoding described in Section II-A. Let

$$x'_d := \sum_{i=0}^{l_x - 1} 2^{B_{x,y} - 1 - i} \cdot s_{x,i} \tag{8}$$

$$y'_d := \sum_{i=0}^{l_y - 1} 2^{B_{x,y} - 1 - i} \cdot s_{y,i} \tag{9}$$

Then the corresponding point is given by

$$\left(x'_d \cdot \frac{360}{C_x} - 180, y_d \cdot \frac{180}{C_y} - 90, 0\right) \tag{10}$$

and the covered volume by

$$\begin{aligned}&\left[x'_d \cdot \frac{360}{C_x} - 180, (x_d + 2^{B_{x,y} - l_x}) \cdot \frac{360}{C_x} - 180\right) \times \\ &\left[y'_d \cdot \frac{180}{C_y} - 90, (y_d + 2^{B_{x,y} - l_y}) \cdot \frac{180}{C_y} - 90\right) \times \\ &[D, H)\end{aligned} \tag{11}$$

Something that is certainly worth nothing is that in GeoPKI, intermediate nodes also store a set of certificates. When using a tree representing the geography (which leads to shorted proof sizes as described in Section III), this is almost a requirement. Without this optimization, almost no leaf would be empty as almost all pieces of land on earth are at least claimed by a country. This would require enormous storage costs and result in less efficient queries.

D. Merkle Hash Tree (MHT)

The just described tree is combined with a MHT. As in F-PKI [3], each leaf stores a set of certificates. Let $C_0, C_1, ..., C_{n-1}$ be the sequence of certificates located in a leaf, sorted lexicographically by the SHA-256 hashes of the PEM encoding in ascending order. The leaf's hash value is then defined as

$$H(0||H(C_0)||H(C_1)||...||H(C_n)) \tag{12}$$

where H is the SHA-256 hashing function. The hash value of an empty / non-existent node in the MHT is $H(0)$.

An intermediate node in the tree has two children (two intermediate nodes or two leaves) and in GeoPKI also stores a set of certificates. As before, let $C_0, C_1, ..., C_{n-1}$ be the sequence of certificates located in an intermediate node, sorted lexicographically by the SHA-256 hashes of the PEM encoding in ascending order. Hash h_3 over the list of certificates is defined as

$$h_3 := H(H(C_0)||H(C_1)||...||H(C_n)) \tag{13}$$

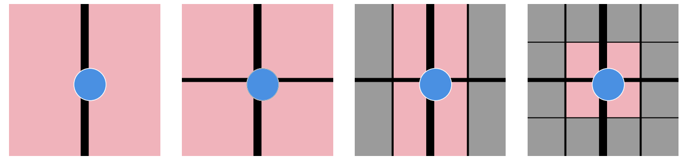
Then let h_1 and h_2 be the hash values of the left and right child of the intermediate node, respectively. The hash value of the intermediate node is then defined as

$$\begin{cases} H(1||h_1||h_2) & \text{if } n = 0 \\ H(1||h_1||h_2||h_3) & \text{if } n > 0 \end{cases} \tag{14}$$

The constants 0 and 1 are prefixed in the hash computation to prevent collisions where a given hash $H(a||b)$ for some a and b could simultaneously refer to a leaf with content $a||b$ or a intermediate node with children hash values a and b.

E. Querying the MHT

Since many sizes of volumes can be represented by the binary format described in Sections II-A and II-C, the only type of query a GeoPKI map server supports is such a binary string. Such a binary string has a unique corresponding node in the tree (that might not exist as the tree is sparse). The result then is the set of all certificates stored at intermediate nodes above the specified node (as these represent larger areas covering the queries point) and the set of all certificates stored in the subtree rooted at the specified node as these are within the queried area. The former set corresponds to the set of nodes whose corresponding binary representation are a prefix for the given query. The latter set corresponds to the set of nodes whose corresponding binary representation the given query is a prefix for.



(a) Areas after one split (b) Areas after two splits (c) Areas after three splits (d) Areas after four splits

Fig. 3: The two-dimensional surface (longitude, latitude) after the first, second, third and fourth split. The blue circle shows a possible area to be claimed by a certificate or the desired query area of a client. The area colored in red is the minimum area that has to be retrieved using the given number of splits.

A trivial way to obtain this set of certificates is to walk the tree from the root and collect the certificates stored at the intermediate nodes along the way. If the next bit is a zero, go left, if it is one, go right. Once the node with a binary string corresponding to the query is found, the full subtree rooted at this node is walked recursively until an empty node or a leaf is found and all found certificates are returned.

F. Arbitrary Volumes and Certificate Placement

The discretization used by GeoPKI does not directly fit all requirements. On one hand, claimed volumes will likely never fit exactly one of the voxels. Moreover, clients likely want to query volumes not aligning with the GeoPKI voxels.

In both cases this volume V has to be approximated using the GeoPKI voxels. A trivial approximation the smallest volume encompassing all of V. Unfortunately this is results in returning the whole tree if V lies at the intersection of one of the first few division lines. Figure 3 depicts this situation with the blue circle corresponding to V and the red area the minimum area that has to be retrieved at the different levels in the tree. Figure 3a shows the line of the first split. Given the blue circle intersects both area, the above trivial approximation would result in the whole tree being returned for a comparable small area that is just placed unfortunately.

Thus a better approximation is required. The main problem with the trivial approximation is the discrepancy of the area queried and the returned area: If the client queries for a large area, it is fine to return a lot of data. But in case of a query for a small area, the size of the returned data should be comparable to the queried area. Given the split lines do not move when going to a deeper level in the tree, we will always have to return multiple voxels. The number of required voxels is monotonically increasing when going deeper down in the tree but the area covered by them monotonically decreases as the voxel approximation for V becomes more accurate. With decreasing area, the data that has to be returned also monotonically decreases as the subtrees contain at most as much data as any of their super-trees. Placing a certificate claiming V in all leaves covered by V is infeasible as the tree will no longer be sparse as most places on earth are claimed by at least one country.

A plausible heuristic balancing the returned data size with the number of subtrees loss in sparsity is to use a subtree level where the volume of V is greater than or equal to a given fraction $f \in [0, 1)$ of the voxel volumes at this level. For instance $f = \frac{1}{3}$ would result in Figure 3d and $f = \frac{1}{2}$ in Figure 3d. The effectiveness of this heuristic and a good value for f is to be determined using empirical experiments.

G. Proofs of Precense and Absence

Given a query for a volume V, the map server should proof that the returned data is in fact in the MHT and that no certificate was omitted. As described in Section II-E, the server directly only supports queries for binary strings that have a unique corresponding node v in the MHT. Let $\mathcal{C}$ with $C \in \mathcal{C}$ be the set of all certificates stored at v. The server has to send the set $\{H(x) \mid \forall x \in \mathcal{C} \backslash \{C\}\}$ to the client as it is required to compute the node's hash (see Equations 12 and 13). As described in Section II-E, if v is an intermediate node, the query result will include the whole subtree rooted at v. Hence a client has all information to compute the hashes of v's children by themselves. These are required to compute the hash of an intermediate node (see Equation 14).

What is left to complete the proof is the set of complementary hashes along the way to the root and the root hash itself as in a standard MHT. Since in GeoPKI the intermediate nodes along the way to the root also have associated certificates, the set of their hashes is also required but as specified in Section II-E, this information is returned to the client anyway and can thus be computed locally.

In a sparse MHT, proving the absence of any certificate in a subtree rooted at v is the same as proving the presence of an empty node. Thus when a client receives a shorted path up to the root than expected, it already has all information required to proof the node's absence in the tree.

Given the binary MHT has 2^B leaves (see Section II), it's depth is B. A proof of presence for a node v at depth $d \in \{0, ..., B-1\}$ (zero is the root itself) then consists of the d complementary node's hash along the path to the root. The size of these hashes is 256 bits or 32 bytes due to the use of SHA-256. Thus in the worst case a proof for a single leaf consists $(B-1) \cdot 32$ B $= 68 \cdot 32$ B $= 2'176$ B ≈ 2 kB and is thus in the same order as an average X.509 certificate. The size of such a proof is smaller the sparser the tree is and is always smaller if an intermediate node, i.e. not a leaf, is queried.

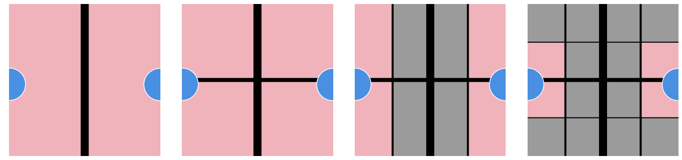
H. Proof of Consistency

As in F-PKI [3] using a consistency tree over the root hashes.

III. Optimizations

First of all it should be noted that the asymmetric design of the binary string with respect to the z dimension is intentional as it allows placing certificates higher in the tree and thus increase sparsity.

Most of the world is rather flat with respect to elevation which means the certificates can often be placed at intermediate nodes covering the full height. This lowers the tree's height and thus reduces the proof sizes. For this to work, clients must not expect the certificates to be placed at a unique node in the tree. The placement does not matter as long as the certificate is stored with *all nodes at one level* whose geometric representation intersect the certificate's claimed volume. This is because a GeoPKI map server always returns the data along the path to the root together with the whole subtree. Hence when the certificate is placed in the subtree of the queried volume it will always be included but if it is placed higher in the tree it will only be included if the certificate is stored at one of the parent nodes.



(a) Areas after one split (b) Areas after two splits (c) Areas after three splits (d) Areas after four splits

Fig. 4: The two-dimensional surface (longitude, latitude) after the first, second, third and fourth split. The blue circle shows a possible area to be claimed by a certificate or the desired query area of a client. The area colored in red is the minimum area that has to be retrieved using the given number of splits. Compare to Figure 3.

Section II-E describes how the map server only serves an answer to a single binary string query. While such a API works, there is room if correlated queries are answered at the same time. For instance when the client derives the different binary strings to query a single volume as described in Section II-F, the queries share the path to the root from their lowest common ancestor. By replying with a single answer for such a set of queries, the data along this shared path has to be returned just once. Given the use of the Z-order curve (see Section II) to order the leaves, points spatially close should also be somewhat close in the tree and thus have a common ancestor on lower levels. Though in the worst case it is still possible that the only common ancestor is the root even if the two areas are spatially close. This happens when the volume covers both sides of the first split done in the binary tree. This scenario is depicted in Figure 3a. It seems impossible to avoid such a worst-case scenario entirely when projecting three dimensions to a single one as in three dimensions each point can have eight neighbors whereas in one dimension it only has two.

An alternative design could use a (hybrid-)quadtree or (hybrid-)octree where most intermediate nodes have four or eight children, respectively. "Hybrid" and "most" because $B_x = B_y > B_z$ which means the volume cannot be subdivided fully using a quad- or octree. Hence when using an octree with eight children, the z dimension is already exhausted after B_z splits and will from there continue to only have four children (i.e. a quadtree). When using a quadtree, the first $B_{x,y}$ splits would solely concern the x and y dimensions and after that the children will be binary trees of height B_z only splitting the z dimension. This will decrease the tree's height but increase the proof size due to the larger number of neighboring hashes that have to be provided on each level. For the hybrid-quadtree, the tree height would be $\log_4(2^{2 \cdot B_{x,y}}) = \log_4(4^{B_{x,y}}) = B_{x,y} = 26$ plus the $\log_2(2^{B_z}) = B_z = 15$ for the binary subtrees. For the hybrid-octree the tree height would be $\log_8(2^{B_z}) = \log_8(8^{B_z/3}) = B_z/3 = 5$ plus the $\log_4(2^{B-B_z}) = \log_4(2^{2 \cdot B_{x,y}}) = B_{x,y} = 26$ for the quad-subtrees. The proof size of a leaf would thus increase to $(26 \cdot (4-1) + (15-1) \cdot (2-1)) \cdot 32$ B $= 2'944$ B ≈ 3 kB and $(5 \cdot (8-1) + (26-1) \cdot (4-1)) \cdot 32$ B $= 3'520$ B ≈ 3.5 kB, respectively. The ones are subtracted from the tree height since the on the last level no complementary hash has to be returned. The ones subtracted from the number of neighbors is due to the fact that the hash resulting from the leaf itself does not have to be provided as it can be computed by the client.

Individual proofs are larger but some spatially close nodes are also "closer" together in the tree due to the reduced height. Still, the worst case scenario remains: Two spatially adjacent volumes intersecting the lines of the first split only share the root node as a common ancestor.

To maintain consistency with the already introduced binary representation of volumes, we can consider the quad-tree part (i.e. the first $B_{x,y}$ levels) to add two bits to the prefix on each level rather than just one. The last B_z levels would then each just add a single bit as before.

It is not clear to what extend the usage a more complex tree structure is beneficial and the simple binary tree is thus preferred for the moment.

Considering that earth is a sphere and that the coordinates wrap around, the situation becomes even more difficult since a similar situation happens at the points where the coordinate system wraps around. This scenario is depicted in Figure 4. This wrap-around can be interpreted as being part of the first split since the problem occurs exactly on the opposite side of the first split when considering the x-axis as wrapping around. To reduce it's impact it would be possible to shift the longitude coordinate system west by $30°$ to this circle around earth (called a *geodesic*) to pass through the Atlantic Ocean and Pacific Ocean rather than central europe.

IV. Caveats

Since longitude (x) and latitude (x) coordinates that are split in half numerically represent polar coordinates on a sphere, numerically equidistant points, areas or volumes are not equidistant on earth. Near the poles the longitudinal circumference is smaller but represented using the same numerically range resulting in higher precision. Hence the discretization range was chosen based on the semi-major axis at the equator where the smallest precision is achieved.

References

[1] NGA geomatics - WGS 84. [Online]. Available: https://earth-info.nga.mil/?dir=wgs84&action=wgs84

[2] S. K. Dhaka and V. Kumar, "Chapter 1 - composition and thermal structure of the earth's atmosphere," in *Atmospheric Remote Sensing*, ser. Earth Observation, A. Kumar Singh and S. Tiwari, Eds. Elsevier, pp. 1–18. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780323992626000237

[3] L. Chuat, C. Krähenbühl, P. Mittal, and A. Perrig, "F-PKI: Enabling innovation and trust flexibility in the HTTPS public-key infrastructure," in *Proceedings 2022 Network and Distributed System Security Symposium*. [Online]. Available: http://arxiv.org/abs/2108.08581